Da pesquisa à entrega automática

Do seu repositório ao ar

Trabalho prático de DevOps
9 capítulos

Conteúdo

01 – Entender o ambiente gratuito (Render Free Tier)

02 – Criar a conta com segurança

03 – Entender a infraestrutura do Render

04 – Criar o serviço e fazer o primeiro deploy

05 – Configurar o serviço de forma mínima e segura

06 – Escolher o app pronto

07 – Publicar o app manualmente

08 – Montar o CI/CD com GitHub + Render

09 – Validar e refletir

01 – Entender o ambiente gratuito (Render Free Tier)

Objetivo

Pesquisar e compreender o que é realmente gratuito no Render Free Tier, quais são os limites e os riscos envolvidos.

O que foi feito

1. Pesquisa da documentação oficial do Render
1. Análise dos recursos do plano Free
1. Identificação das limitações (cold start, horas mensais)
1. Comparação com Oracle Cloud Free Tier e outras opções

Resumo do Free Tier do Render

O Render oferece um plano Free permanente com as seguintes características:

1. Web Services gratuitos
1. 750 horas de instância por mês (suficiente para 1 serviço 24/7)
1. SSL automático
1. Deploy automático a partir do GitHub
1. Não exige cartão de crédito na maioria dos casos

Limitações importantes

1. O serviço dorme após 15 minutos sem tráfego
1. Ao receber uma nova requisição, demora cerca de 30–60 segundos para “acordar” (cold start)
1. 512 MB de RAM e 0.1 CPU no plano Free

Tabela dos recursos Free

Web Service	1 instância	Dorme após 15 min idle
Horas de instância	750 horas/mês	Suficiente para 1 app 24/7
RAM	512 MB	
CPU	0.1	
Build minutes	500/mês	
Bandwidth	~100 GB/mês	
SSL	Grátis e automático	
Deploy do GitHub	Sim, automático	

Riscos e como evitá-los

1. Ultrapassar as 750 horas (improvável com 1 serviço)
1. Exceder bandwidth (raro em trabalho acadêmico)
1. Dependência do cold start (documentar essa limitação)

Comparação com Oracle Cloud Always Free

Cartão necessário	Geralmente não	Sim
SSH / controle total	Não	Sim
Cold start	Sim (15 min)	Não
Facilidade de deploy	Muito alta	Média
Recursos de compute	Baixos	Altos (2 OCPU / 12 GB)
Ideal para	Apps web simples + CI/CD	Servidor completo + aprendizado

Evidências

Evidência visual não incluída no repositório: Página oficial do Render Free (../images/01-render-free.png)

Fontes consultadas

1. Render Documentation
1. Render Pricing
1. Uso de IA: Grok (xAI)

Conclusão do capítulo

O Render Free Tier atende perfeitamente ao objetivo do trabalho (app no ar + deploy automático), com a vantagem de não exigir cartão. As limitações (principalmente o cold start) foram compreendidas e serão documentadas.

02 – Criar a conta com segurança

Objetivo

Criar a conta no Render de forma segura, conectando apenas o necessário (GitHub) e entendendo as permissões concedidas.

O que foi feito

1. Criação da conta no Render usando login com GitHub
1. Verificação das permissões concedidas ao Render no GitHub
1. Confirmação de que não foi solicitado cartão de crédito

Região / Local escolhido

1. Conta criada com login via GitHub
1. Serviço posteriormente implantado na região disponível do plano Free

Medidas de segurança ativadas

1. Autenticação via GitHub (OAuth)
1. Não foi necessário cadastrar cartão
1. Permissões do Render limitadas ao repositório necessário

Evidências

Conta criada no Render
Permissões do GitHub

Problemas encontrados e soluções

1. Nenhum problema. O processo foi direto e não solicitou cartão.

Fontes consultadas

1. Documentação do Render
1. Uso de IA: Grok (xAI)

Conclusão do capítulo

A conta foi criada com sucesso usando o GitHub como provedor de identidade, sem necessidade de cartão de crédito. O ambiente está pronto para o deploy.

03 – Entender a infraestrutura do Render

Objetivo

Compreender como o Render organiza a infraestrutura (em contraste com VCN, subnet e security list do Oracle Cloud) e quais controles de rede existem no plano Free.

Conceitos explicados com minhas palavras

Web Service

É o equivalente a uma “instância” no Render. Você define o runtime (Node, Python, etc.), o comando de start e o Render cuida do restante.

Deploy automático

A cada push no repositório conectado, o Render detecta a mudança, faz o build e publica uma nova versão automaticamente.

Cold Start

Como o plano Free coloca o serviço para dormir após 15 minutos sem tráfego, a primeira requisição depois disso demora alguns segundos para “acordar” o container.

SSL / HTTPS

O Render fornece certificado SSL gratuito e automático para todos os serviços.

Domínio

Todo serviço recebe um subdomínio `*.onrender.com`. É possível adicionar domínio próprio depois.

Diagrama da infraestrutura

```
GitHub (push)
     |
     v
Render Build
     |
     v
Web Service (container)
     |
     +---> HTTPS (SSL automático)
     |
     v
Internet (usuários)
```

Regras de “rede” / exposição

Porta exposta	Definida pelo Start Command (3000)
HTTPS	Obrigatório e automático
Firewall	Gerenciado pelo Render
Acesso SSH	Não disponível no Free
IP público fixo	Não (usa load balancer do Render)

Evidências

Evidência visual não incluída no repositório: Dashboard do serviço no Render (../images/03-render-dashboard.png)

Fontes consultadas

1. Documentação oficial do Render
1. Uso de IA: Grok (xAI)

Conclusão do capítulo

Embora o Render não exponha VCN, Security List ou SSH, a infraestrutura é suficiente e muito mais simples para o objetivo do trabalho: ter o app no ar com deploy automático.

04 – Criar o serviço e fazer o primeiro deploy

Objetivo

Criar o Web Service no Render e realizar o primeiro deploy a partir do repositório GitHub.

Configuração escolhida e justificação

Tipo de serviço	Web Service	Adequado para aplicação Node.js
Runtime	Node	App escrito em Node.js
Root Directory	`app`	Código fica dentro da pasta app/
Build Command	`npm install`	Instala dependências
Start Command	`node server.js`	Inicia o servidor
Plano	Free	Atende ao requisito de ambiente gratuito

Acesso ao serviço

Diferente do Oracle Cloud, não existe SSH. O acesso é feito exclusivamente via HTTPS pelo link gerado pelo Render.

URL do serviço: https://devops-app-x1r7.onrender.com/

Evidências

Serviço criado e em execução
Primeiro deploy com sucesso
App respondendo no navegador

Problemas encontrados e soluções

1. Nenhum problema relevante. O deploy foi bem-sucedido na primeira tentativa.

Fontes consultadas

1. Documentação do Render (Web Services)
1. Uso de IA: Grok (xAI)

Conclusão do capítulo

O serviço foi criado e o aplicativo está acessível publicamente. O próximo passo é garantir que o ambiente esteja mínimo e documentar o processo.

05 – Configurar o serviço de forma mínima e segura

Objetivo

Manter o serviço o mais simples possível, usando apenas o necessário e aproveitando as proteções nativas do Render.

O que foi configurado

Runtime	Node	App é Node.js
Porta	3000 (padrão do app)	Render detecta automaticamente
HTTPS	Automático	Fornecido pelo Render
Variáveis de ambiente	Nenhuma sensível	App não precisa de secrets
Auto-Deploy	Ativado	Deploy a cada push

Comandos / configuração usada

No painel do Render:

1. Root Directory: `app`
1. Build Command: `npm install`
1. Start Command: `node server.js`

No código do app:

```
const PORT = process.env.PORT || 3000;
```

O Render injeta a variável `PORT` automaticamente.

Proteções nativas aproveitadas

1. HTTPS obrigatório
1. Isolamento de container
1. Deploy apenas a partir do repositório autorizado
1. Logs disponíveis no painel

Evidências

Configuração do serviço
Resposta HTTP do app

Fontes consultadas

1. Documentação do Render
1. Uso de IA: Grok (xAI)

Conclusão do capítulo

O serviço está configurado de forma mínima e segura, aproveitando os recursos nativos do Render sem adicionar complexidade desnecessária.

06 – Escolher o app pronto

Objetivo

Escolher um aplicativo simples, com licença permissiva, adequado para rodar no Render Free Tier.

Nome, origem e licença do app

1. Nome: devops-app
1. Origem: criado neste repositório (`app/`)
1. Licença: MIT

Motivo da escolha

1. Extremamente leve (Node.js puro, sem dependências pesadas)
1. Roda perfeitamente no plano Free do Render
1. Fácil de entender e de modificar
1. Permite demonstrar deploy automático com clareza
1. Licença MIT permite uso livre

Requisitos para rodar

Runtime	Node.js 18+
Porta	3000 (ou process.env.PORT)
Banco de dados	Nenhum
Variáveis de ambiente	Nenhuma obrigatória
Memória	Muito baixa (< 50 MB)

Plano de publicação

1. Código já está no repositório
2. Conectar o repositório ao Render
3. Definir Root Directory = `app`
4. Build: `npm install`
5. Start: `node server.js`

Evidências

Código do app no repositório
Licença MIT

Fontes consultadas

1. Este repositório

1 Uso de IA: Grok (xAI)

Conclusão do capítulo

Foi escolhido um aplicativo mínimo, leve e com licença permissiva, ideal para o ambiente gratuito do Render.

07 – Publicar o app manualmente

Objetivo

Registrar o processo de publicação manual (primeiro deploy) antes de confiar totalmente no deploy automático.

Passo a passo manual completo

1. Acessar https://render.com e fazer login com GitHub
2. Clicar em New + → Web Service
3. Selecionar o repositório `LuizzHSA/Trabalho-DEVOPS`
4. Configurar:
    1. Name: `devops-app`
    1. Root Directory: `app`
    1. Runtime: Node
    1. Build Command: `npm install`
    1. Start Command: `node server.js`
    1. Instance Type: Free
5. Clicar em Create Web Service
6. Aguardar o build e o deploy
7. Acessar a URL gerada

Como o app sobrevive a reinícios

O Render gerencia o ciclo de vida do container. Mesmo que o serviço durma (cold start), ele é reativado automaticamente na próxima requisição. Não é necessário pm2 ou systemd.

Endereço público

https://devops-app-x1r7.onrender.com/

Evidências

Deploy manual concluído
App acessado publicamente

Fontes consultadas

1. Documentação do Render
1. Uso de IA: Grok (xAI)

Conclusão do capítulo

O aplicativo foi publicado com sucesso e está acessível publicamente. O processo manual foi compreendido e documentado.

08 – Montar o CI/CD com GitHub + Render

Objetivo

Configurar o deploy automático de forma que todo push na branch main atualize o aplicativo no Render sem intervenção manual.

Link do repositório

https://github.com/LuizzHSA/Trabalho-DEVOPS

Diagrama do fluxo

```
Desenvolvedor
     |
  git push (main)
     |
     v
GitHub Repository
     |
     v
Render (auto-deploy ativado)
     |
     +---> Build (npm install)
     +---> Start (node server.js)
     |
     v
App atualizado em produção
https://devops-app-x1r7.onrender.com/
```

Como o CI/CD funciona no Render

O Render possui integração nativa com GitHub. Ao conectar o repositório e ativar o Auto-Deploy, qualquer push na branch configurada dispara automaticamente:

1. Clone do código
2. Execução do Build Command
3. Execução do Start Command
4. Substituição da versão anterior (zero-downtime quando possível)

Não foi necessário criar workflow manual no `.github/workflows` para o deploy principal, pois o próprio Render faz esse papel.

Segredos e proteção

1. A conexão é feita via OAuth do GitHub
1. O Render só tem acesso ao repositório autorizado
1. Não há chaves SSH expostas no repositório

Evidências

Auto-Deploy ativado no Render
Deploy disparado por push
App antes do push
App depois do push

Fontes consultadas

1 Documentação do Render (Auto-Deploy)

1 Uso de IA: Grok (xAI)

Conclusão do capítulo

O pipeline de CI/CD está funcionando. Qualquer alteração enviada para a branch main é automaticamente publicada no Render.

09 – Validar e refletir

Objetivo

Comprovar que os três objetivos principais foram alcançados e fazer uma reflexão final.

Checklist dos três objetivos

App rodando em ambiente próprio (Render)	✅	https://devops-app-x1r7.onrender.com/
Deploy automático a cada push no GitHub	✅	Auto-Deploy do Render ativado
Documentação completa em 9 capítulos	✅	Este repositório

Limitações do ambiente gratuito (Render)

1. Cold start de 30–60 segundos após 15 minutos de inatividade
1. Recursos limitados (512 MB RAM / 0.1 CPU)
1. Sem acesso SSH / shell
1. Sem IP público fixo
1. 750 horas de instância por mês

Riscos de segurança que restam e melhorias possíveis

Cold start	Plano pago (sempre ligado)
Recursos baixos	Upgrade de instância
Sem SSH	Usar plataforma com VPS (Oracle etc.)
Dependência de um único provedor	Adicionar domínio próprio + monitor

Aprendizados pessoais

Durante o trabalho foi possível vivenciar o ciclo completo de publicação de um aplicativo: pesquisa de ambientes gratuitos, criação de conta, escolha do app, publicação manual e, principalmente, a automatização do deploy.

A troca do Oracle Cloud pelo Render (devido à necessidade de cartão) mostrou que existem caminhos alternativos viáveis para cumprir os mesmos objetivos pedagógicos, mesmo com trade-offs diferentes (simplicidade vs controle total).

Fontes consultadas

1. Todos os capítulos anteriores

1. Documentação do Render
1. Uso de IA: Grok (xAI) para orientação e estruturação de todo o trabalho

Conclusão final

Os três objetivos foram cumpridos:

1. O app está no ar
2. O deploy acontece automaticamente a cada push
3. Todo o processo está documentado em nove capítulos

Trabalho concluído com sucesso.